

Time-Space Tradeoffs of Truncation with Preprocessing

Krzysztof Pietrzak*

Pengxiang Wang†

April 22, 2025

Abstract

Truncation of cryptographic outputs is a technique that was recently introduced in Baldimtsi et al. [BCKK22]. The general idea is to try out many inputs to some cryptographic algorithm until the output (e.g. a public-key or some hash value) falls into some sparse set and thus can be compressed: by trying out an expected 2^k different inputs one will find an output that starts with k zeros.

Using such truncation one can for example save substantial gas fees on Blockchains where storing values is very expensive. While [BCKK22] show that truncation preserves the security of the underlying primitive, they only consider a setting without preprocessing. In this work we show that lower bounds on the time-space tradeoff for inverting random functions and permutations also hold with truncation, except for parameters ranges where the bound fails to hold for “trivial” reasons.

Concretely, it’s known that any algorithm that inverts a random function or permutation with range N making T queries and using S bits of auxiliary input must satisfy $S \cdot T \geq N \log N$. This lower bound no longer holds in the truncated setting where one must only invert a challenge from a range of size $N/2^k$, as now one can simply save the replies to all $N/2^k$ challenges, which requires $S = \log N \cdot N/2^k$ bits and allows to invert with $T = 1$ query.

We show that with truncation, whenever S is somewhat smaller than the $\log N \cdot N/2^k$ bits required to store the entire truncated function table, the known $S \cdot T \geq N \log N$ lower bound applies.

1 Introduction

1.1 Truncation

In [BCKK22] Baldimtsi, Chalkias, Chatzigiannis and Kelkar suggested *truncation* as a technique to shorten outputs of cryptographic primitives like hash values, signatures or public keys. The technique applies whenever there’s a part of the input that can be chosen arbitrarily, like the randomness in signing and key generation or parts of the input when hashing. The general idea is to simply try out many different inputs until the output lands in some sparse domain, say it starts with Δ zeros, and such an output can be encoded using Δ less bits than a general output.

While the applicability of this technique is limited by the fact that finding an output that starts with Δ zeros will require around 2^Δ invocations of the primitive (so e.g., compressing by 20 bits already requires around a million invocations), in [BCKK22] interesting applications are identified where this can make a big difference, including in the context of the Ethereum blockchain, where saving a few bits to be stored on-chain can lead to significant savings. We discuss another application to proofs of space as used in the Chia network in the open problems Section 6.

While truncation puts extra burden on the evaluator (say, a signer), there’s no extra cost for the parties that use the output (say, verify the signature). Moreover [BCKK22] show that truncation preserves security in the bit security framework of [WY21]. Informally, a primitive has bit security κ , if every adversary who breaks the security with advantage ϵ must run in time $2^\kappa \cdot \epsilon$.

*Institute of Science and Technology Austria

†École Polytechnique Fédérale de Lausanne

1.2 Time-Space Tradeoffs.

While the preservation of bit security under truncation gives some confidence in this technique, it only considers a setting without precomputation. Unfortunately, truncation can decrease the security of primitives when the adversary is given some auxiliary input. Let us illustrate this considering the one-wayness of a permutation over n bits, which we'll denote with $f : [N] \rightarrow [N]$ where $N = 2^n$. A random permutation f does have n bits of bit security, i.e., given a random $y \in [N]$, finding $x, f(x) = y$ will require $N/2$ invocations to f in expectation. But given S bits of advice (that depend on f but not the challenge y), it's possible to invert f using just T invocations whenever

$$S \cdot T \geq N \log(N) \quad (1)$$

The general idea is to compute and store values $x_0, x_T, x_{2T}, \dots$ where $x_i = f(x_{i-1})$. On challenge y one now applies f until one of the stored $x_{i \cdot T}$ values is hit, and then continues to apply f to $x_{(i-1)T}$ until one hits y . For functions the best space-time tradeoff are somewhat worse, for *random* functions an attack with

$$S^2 \cdot T \in \Theta(N^2 \log(N))$$

is achieved by Hellman tables [Hel80] or rainbow tables [Oec03]. So, any permutation can be inverted with time and space, e.g. $T = S \approx N^{1/2}$ while *random* functions can be inverted with $T = S \approx N^{2/3}$ (for functions that are not random the existing bounds are somewhat worse [FN91]).

De, Trevisan and Tulsiani [DTT10] (building on work by Yao [Yao90], Gennaro-Trevisan [GT00] and Wee [Wee05]) prove that the attack as in eq.(1) is basically optimal as any adversary who inverts a *random* permutation or function with a range of size N must satisfy

$$S \cdot T \in \Omega(N) \quad (2)$$

Note that the above is seemingly wrong if $T = 0$ or $S = 0$ as one can invert with no space but N queries or $N \log N$ space and no queries. The proof of the lower bound assumes adversary must always query f on its output, so T is at least 1, and S is assumed to be at least $\log N$ which is required to store the challenge.

The same lower bound applies for random *functions*, though note that in this case it's not matching the upper bound. The actual lower bound is slightly more general showing that $S \cdot T \in \Omega(\epsilon \cdot N)$ must hold for any adversary who inverts with some probability ϵ , but for this introduction we assume ϵ is some constant.

1.3 Time-Space Tradeoffs with Truncation

Now consider the truncated setting, where we have a random function or permutation $f : [N] \rightarrow [N]$, but must only invert it on outputs sampled from some truncated range, say where the first Δ bits are set to zero. We'll denote this range with $[\delta N]$ where $\delta = 2^{-\Delta}$.

Now one can store the δN preimages of f on $[\delta N]$ using $S = \delta N \log N$ bits, and using this advice, it's possible to find the preimage $x, f(x) = y$ for any $y \in [\delta N]$ without invoking f at all. As outlined above, this means $T = 1$, but it still contradicts the lower-bound from eq.(2) as $\delta N \log N \notin \Omega(N)$ for $\delta \in o(1/\log N)$.

The main result in this work is Theorem 3.2, which basically states that the issue just described is the only reason for the lower bound to fail: as long as S is too small to basically store the entire function table of the *truncated* function, the $S \cdot T \in \Omega(N)$ lower bound applies. While we state and prove the bound for functions, it can easily be adapted to permutations, we discuss other truncated primitive in the open problems Section 6.

The technical result implying Theorem 3.2 is stated in Lemma 3.3. It uses a so called compression argument: it shows how an adversary that inverts a random function f can be turned into an encoding for the function table of f , where the length of the encoding depends on the space and time efficiency of the adversary. As the function table of a random function is incompressible, as stated in Fact 1, we can derive a lower bound on the space and time complexity of the adversary.

1.4 The Compression Argument

The starting point for proving the Lemma is a proof of the $S \cdot T \geq N \log N$ lower bound for inverting random functions from [AAC⁺17a]. This proof is less elegant than the proof of De et al. from [DTT10], which uses a high level argument about sets, while [AAC⁺17a] provide pseudocode of the encoding and argue about the length of its output.

Their encoding is basically Algorithm 2 in this paper for $\delta = 1$. It takes as input an adversary $\mathcal{A}$ who is guaranteed to invert some function $f : [N] \rightarrow [N]$ on an ϵ fraction of the range and making no more than T queries.

$\mathcal{A}$ is then invoked on some challenges, and every time $\mathcal{A}$ inverts a challenge we get one entry in the function table “for free”. With every invocation, $\mathcal{A}$ can make up to T queries to f , and those queries can later no longer be used as challenges, i.e., we “spoil” up to T of the $\epsilon \cdot N$ challenges with every successful inversion. Overall we can get $\mathcal{A}$ to invert something in the order of $\epsilon \cdot N/T$ challenges before running out of unspoiled challenges, and thus get an encoding that is around $\epsilon \cdot N/T$ bits shorter than the function table. As a random function is incompressible, this then implies that the advice used by $\mathcal{A}$ must be around $S \gtrsim \epsilon \cdot N/T$. That’s how the $S \cdot T \in \Omega(\epsilon \cdot N)$ lower bound in [AAC⁺17a] is proven.

In the truncated setting when $\delta \ll 1$, $\mathcal{A}$ is only guaranteed to invert on an ϵ fraction of the sparse domain $[\delta N]$, and the above argument would only gives us a $S \gtrsim \delta \cdot \epsilon \cdot N/T$ bound.

A key observation is the fact that we get a $S \gtrsim \epsilon \cdot N/T$ lower bound even in the truncated case if we assume that not much more than a δ fraction of the queries made by $\mathcal{A}$ map to the sparse $[\delta \cdot N]$ domain, as then each compressed entry only spoils around $\delta \cdot T$ of the $[\epsilon N]$ available challenges.

To prove our lemma we now make a case distinction. If for every compressed value we typically do not spoil more than $T_g \leq 12\delta T$ potential challenges, we use the observation above, so in this case we use basically the same argument as in [AAC⁺17a].

In the other case the queries made by $\mathcal{A}$ fall into the sparse $[\delta N]$ set at least 12 times more often than a random query would. Knowing that the output $f(x)$ on a query x is in $[\delta N]$ means we can encode it using just $\log N - \log 1/\delta$ bits. We will encode the positions of the queries made by $\mathcal{A}$ that fall into $[\delta N]$, there are around $\epsilon \delta N$ such queries, and then encode each output using just $\log N - \log 1/\delta$ bits. As the queries that fall into $[\delta N]$ are sufficiently dense (i.e., 12 times denser than a random query would), encoding those positions uses a bit less per entry than the $\log 1/\delta$ bits we save knowing the entry is in $[\delta N]$. So overall we save $\epsilon \delta N$ bits, and thus for this case can conclude (again using the incompressibility of a random function table) that the space used by $\mathcal{A}$ must be at least $S \gtrsim \epsilon \delta N$.

2 Notation and Basic Facts

We use brackets like $(x_1, x_2, \dots)$ or $\{x_1, x_2, \dots\}$ to denote ordered sets (aka. lists) and unordered sets, respectively. $[N]$ denotes some domain of size N , for notational convenience we assume $N = 2^n$ is a power of two and identify $[N]$ with $\{0, 1\}^n$. For some $\Delta \leq n$ and $\delta = 2^{-\Delta}$, we’ll denote with $[\delta N]$ some subset of $[N]$ of size δN (the truncated range), say $0^\Delta \| 0^{n-\Delta}$, the set of n bits strings that start with Δ zeros, but in principle any subset whose elements can be compressed to (not much more than) $n - \Delta$ bits will do.

For a function $f : [N] \rightarrow [M]$ and a set $S \subseteq [N]$, we denote with $f(S)$ the set $\{f(S[1]), \dots, f(S[|S|])\}$, similarly for a list $L \subseteq [N]$, $f(L)$ is the list $(f(L[1]), \dots, f(L[|L|]))$.

Randomized adversaries are treated as if they were deterministic. However, this is w.l.o.g. as they are only invoked within encoding/decoding procedures that have access to a shared randomness that can be used to derandomize them.

The following well known fact captures the fact that one cannot compress a random string.

Fact 1 (from [DTT10]) *For any randomized encoding procedure $\text{Enc} : \{0, 1\}^r \times \{0, 1\}^n \rightarrow \{0, 1\}^m$ and decoding procedure $\text{Dec} : \{0, 1\}^r \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ where*

$$\Pr_{x \leftarrow \{0, 1\}^n, r \leftarrow U_{|r|}} [\text{Dec}(r, \text{Enc}(r, x)) = x] \geq \delta$$

we have $m \geq n - \log(1/\delta)$

Fact 2 *If a set X is at least ϵ dense in Y , i.e., $X \subset Y$, $|X| \geq \epsilon|Y|$, and Y is known, then X can be encoded using $|X| \cdot \log(e/\epsilon)$ additional bits. To show this we use the inequality $\binom{n}{\epsilon n} \leq (en/\epsilon n)^{\epsilon n}$, which implies $\log \binom{n}{\epsilon n} \leq \epsilon n \log(e/\epsilon)$.*

3 Main Theorem and Lemma

De, Trevisan and Tulsiani [DTT10] show that the simple time-space tradeoff eq.(1) for inverting permutations is basically tight.

Theorem 3.1 ([DTT10], as stated in [AAC⁺17a]) *Fix some $\epsilon \geq 0$ and an oracle algorithm $\mathcal{A}_{\text{aux}}$ that takes an advice string aux of length $|\text{aux}| = S$ and makes at most T oracle queries. If for every function $f : [N] \rightarrow [N]$ there exists a string aux such that*

$$\Pr_{y \leftarrow [N]} [f(\mathcal{A}_{\text{aux}}^f(y)) = y] \geq \epsilon$$

then

$$T \cdot S \in \Omega(\epsilon N) . \quad (3)$$

In this work we show that this result extends to the case where the challenge comes from a sparse set $[\delta N]$

Theorem 3.2 (main) *Fix some $\epsilon \geq 0$ and an oracle algorithm $\mathcal{A}_{\text{aux}}$ that takes an advice string aux of length $|\text{aux}| = S$ and makes at most T oracle queries. If for every function¹ $f : [N] \rightarrow [N]$ there exists a string aux such that*

$$\Pr_{y \leftarrow [\delta N]} [f(\mathcal{A}_{\text{aux}}^f(y)) = y] \geq \epsilon$$

then either

$$S \in \Omega(\delta \epsilon N) \quad \text{or} \quad T \cdot S \in \Omega(\epsilon N) . \quad (4)$$

The theorem is proven using a compression argument as stated in the following lemma.

Lemma 3.3 (generalized version of a Lemma from [AAC⁺17a]) *Let $\mathcal{A}_{\text{aux}}, T, S, \epsilon$ and f be as Theorem 3.2, and assume $T \leq \delta \epsilon N / 40$. There are randomized encoding and decoding procedures Enc, Dec such that if $f : [N] \rightarrow [N]$ is a function and for some $\text{aux}, |\text{aux}| = S$*

$$\Pr_{y \leftarrow [\delta N]} [f(\mathcal{A}_{\text{aux}}^f(y)) = y] \geq \epsilon$$

then

$$\Pr_{r \leftarrow U_{|r|}} [\text{Dec}(r, \text{Enc}(r, \text{aux}, f)) = f] \geq 0.9 \quad (5)$$

and the length of $\text{Enc}(r, \text{aux}, f)$ is at most

$$\underbrace{N \log N}_{=|f|} - \frac{\epsilon \delta N}{2T_g} + S + \log(N) \quad (6)$$

Moreover for some $T_g, 1 \leq T_g \leq T$ which is defined by the encoding algorithm: if $T_g \geq 12\delta T$, we can improve the length of the encoding to

$$\underbrace{N \log N}_{=|f|} - \frac{\epsilon \delta N}{2T_g} + S + \log(N) - \delta \epsilon N \quad (7)$$

¹The proof shows something stronger, and “every function” can be relaxed to “with non-negligible probability for a random function or permutation”.

The T_g above is the average number of f queries that land in the sparse set (i.e., x s.t. $f(x) \in [\delta N]$) that $\mathcal{A}_{\text{aux}}^f$ (when invoked by **Enc** as defined by Algorithm 2 below) makes for every value it inverts. As $\mathcal{A}$ makes at most T queries per challenge we have $T_g \leq T$. For a random query $x \in [N]$ we have $f(x) \in [\delta N]$ with probability δ , so if T_g is significantly larger than δT , this means that the queries made by $\mathcal{A}_{\text{aux}}$ are special in the sense that they map to $[\delta N]$ much more often than random queries would. We use this crucial observation for a case distinction, deriving the left or the right hand side of eq.(4), depending on whether T_g is below or above $12\delta T$.

4 How Theorem 3.2 follows from Lemma 3.3

4.1 Proof of Thm. 3.2 if $T_g \leq 12\delta T$

If $T_g \leq 12\delta T$, the theorem follows from Lemma 3.3 using Fact 1 as follows: assume the function table of f in the lemma is chosen uniformly at random (i.e., x in Fact 1 is a uniform $N \log N$ bit string), then the term in Eq.(6) can be lower bounded as

$$\underbrace{N \log N}_{=|f|} - \frac{\epsilon \delta N}{2T_g} + S + \log(N) \geq N \log N - \log(1/0.9)$$

Reordering we get

$$S \geq \frac{\epsilon \delta N}{2T_g} - \log(N) - \log(1/0.9)$$

using our assumption that $T_g \leq 12\delta T$

$$S \geq \frac{\epsilon N}{24T} - \log(N) - \log(1/0.9)$$

$$T \cdot S \geq \frac{\epsilon N}{24} - T \cdot \log(N) - T \cdot \log(1/0.9)$$

So $T \cdot S \in \Omega(\epsilon N)$ as claimed (on the lhs of eq.(4) in the Theorem). Note that the extra assumption that $T \leq \epsilon N/40$ in the lemma doesn't matter, as if it's not satisfied the theorem is trivially true.

4.2 Proof of Thm. 3.2 if $T_g > 12\delta T$

If $T_g > 12\delta T$, the theorem again follows from Lemma 3.3 using Fact 1, i.e., we again assume the function table of f in the lemma is chosen uniformly at random (i.e., x in Fact 1 is a uniform $N \log N$ bit string), and now the term in Eq.(7) can be lower bounded as

$$\underbrace{N \log N}_{=|f|} - \frac{\epsilon \delta N}{2T_g} + S + \log(N) - \delta \epsilon N \geq N \log N - \log(1/0.9)$$

Note that, as in this case we use eq.(7) rather than eq.(6), we have an extra $-\delta \epsilon N$ term on the lhs. Reordering we get

$$S \geq \frac{\epsilon \delta N}{2T_g} - \log(N) - \log(1/0.9) + \delta \epsilon N$$

and thus $S \in \Omega(\delta \epsilon N)$ as claimed.

5 Proof of Lemma 3.3

We always assume that if $A_{\text{aux}}^f(y)$ outputs some value x , it makes the query $f(x)$ at some point. This is basically w.l.o.g. as we can turn any adversary into one satisfying this by making at most one extra query. If at some point $A_{\text{aux}}^f(y)$ makes an oracle query x where $f(x) = y$, then we also w.l.o.g. assume that right after this query A outputs x and stops.

Algorithm 1 Enc

- 1: **Input:** $\mathcal{A}$, aux , randomness r and a function $f : [N] \rightarrow [N]$ to compress (using that $\mathcal{A}_{\text{aux}}^f(\cdot)$ inverts f on some ϵ fraction of $[\delta N]$)
 - 2: Initialize: $B, G := \emptyset, c := -1$
 - 3: Throughout we identify $[N]$ with $\{0, \dots, N-1\}$ and $[\delta N]$ with $\{0, \dots, \delta N-1\}$
 - 4: Pick a random permutation $\pi : [\delta N] \rightarrow [\delta N]$ (using random coins r)
 - 5: Let $J := \{y \in [\delta N] : f(\mathcal{A}_{\text{aux}}^f(y)) = y\}$, $|J| = \epsilon \delta N$ ▷ The set $J \subset [\delta N]$ where $\mathcal{A}$ inverts
 - 6: For $i = 0, \dots, \delta N-1$ define $y_i := \pi(i)$. ▷ Randomize the order
 - 7: For $y \in J$ let $q(y)$ denote all queries made by $A^f(y)$ except the last query (which is x s.t. $f(x) = y$).
 - 8: **while** $|B| < |J|/2 (= \epsilon \delta N/2)$ **do** ▷ While the bad set contains less than half of J
 - 9: $c := \min\{c' > c : y_{c'} \in \{J \setminus B\}\}$ ▷ Increase c to the next y_c in $J \setminus B$
 - 10: $G := G \cup y_c$ ▷ Add this y_c to good set
 - 11: $B := B \cup (f(q(y_c)) \cap J)$ ▷ Add spoiled queries to bad set
 - 12: **end while**
 - 13: Let $G = \{g_1, \dots, g_{|G|}\}$, $Q = q(g_1), \dots, q(g_{|G|})$, and define $Q' = (q'_1, \dots, q'_{|G|})$, $q'_i \subseteq q(g_i)$ to contain only the “fresh” queries in Q by deleting all but the first occurrence of every element. E.g. if $(q(g_1), q(g_2)) = ((1, 2, 3, 1), (2, 4, 5, 4))$ then $(q'_1, q'_2) = ((1, 2, 3), (4, 5))$.
 - 14: Let $G^{-1} = \{\mathcal{A}_{\text{aux}}^f(y) : y \in G\}$
 - 15: Output an encoding of (the set) G , (the lists) $f(Q'), (|q'_1|, \dots, |q'_{|G|}|)$, (the remaining outputs) $f([N] - \{G^{-1} \cup Q'\})$ and (the advice string) aux . ▷ We discuss how exactly this encoding looks in the proof. The encoding of $f(Q')$ will depend on the average number of spoiled queries per challenge $T_g = |B|/|G|$ ($\approx \epsilon \delta N/2|G|$)
-

Algorithm 2 Dec

- 1: **Input:** $\mathcal{A}, r$ and the encoding $(G, f(Q'), (|q'_1|, \dots, |q'_{|G|}|), f([N] - \{G^{-1} \cup Q'\}), \text{aux})$.
 - 2: Let π be as in Enc.
 - 3: Let $(g_1, \dots, g_{|G|})$ be the elements of G ordered as they were added by Enc (i.e., $\pi^{-1}(g_i) < \pi^{-1}(g_{i+1})$ for all i).
 - 4: Invoke $\mathcal{A}_{\text{aux}}^{(\cdot)}$ sequentially on inputs $g_1, \dots, g_{|G|}$ using $f(Q')$ to answer $\mathcal{A}_{\text{aux}}$'s oracle queries.
 - 5: Combine the mapping $G^{-1} \cup Q' \rightarrow f(G^{-1} \cup Q')$ (which we learned in the previous step) with $[N] - \{G^{-1} \cup Q'\} \rightarrow f([N] - \{G^{-1} \cup Q'\})$ to learn the entire $[N] \rightarrow f([N])$
 - 6: Output $f([N])$
-

5.1 The Size of the Encoding

We will now upper bound the size of the encoding of $G, f(Q'), (|q_1|, \dots, |q_{|G|}|), f([N] - \{G^{-1} \cup Q'\})$ as output in line (15) of the Enc algorithm.

Let $T_g := |B|/|G|$ be the average number of elements we added to the bad set B for every element added to the good set G , then

$$|G| \geq N\epsilon\delta/2T_g. \quad (8)$$

To see this we note that when we leave the while loop (see line (8) of the algorithm Enc) it holds that

$$|B| \geq |J|/2 = \epsilon\delta N/2 \text{ so } |G| = |B|/T_g \geq |J|/2T_g = N\epsilon\delta/2T_g \quad (9)$$

- G : Instead of G we will actually encode the set $\pi^{-1}(G) = \{c_1, \dots, c_{|G|}\}$, from this decoding Dec (who gets r , and thus knows π) can then reconstruct $G = \pi(\pi^{-1}(G))$. We claim that the elements in $c_1 < c_2 < \dots < c_{|G|}$ are whp. at least $\epsilon\delta/2$ dense in $[c_{|G|}]$ (equivalently, $c_{|G|} \leq 2|G|/\epsilon\delta$). By Fact 2 we can thus encode $\pi^{-1}(G)$ using $|G| \log(2e/\epsilon\delta) + \log N$ bits (the extra $\log N$ bits are used to encode the size of G which is required so decoding later knows how to parse the encoding). To see that the c_i 's are $\epsilon\delta/2$ dense whp. consider line (9) in Enc which states $c := \min\{c' > c : y_{c'} \in \{J \setminus B\}\}$. If we

replace $J \setminus B$ with J , then the c_i 's would be whp. close to $\epsilon\delta$ dense in $[N]$ as $J \subset N$ has size $\epsilon\delta N$ and the y_i are uniformly random. As $|B| < |J|/2$, using $J \setminus B$ instead of J will decrease the density by at most a factor 2. If we don't have this density, i.e., $c_{|G|} > 2|G|/\epsilon\delta$, we consider encoding to have failed.

$(|q'_1|, \dots, |q'_{|G|}|)$: Require $|G| \log T$ bits as each $q'_i \leq q_i \leq T$. A more careful argument (using Fact 2 and that the q'_i are on average at most T_g) requires $|G| \log(eT_g)$ bits.

$f([N] - \{G^{-1} \cup Q'\})$: Requires $(N - |G| - |Q'|) \log N$ bits (using that $G^{-1} \cap Q' = \emptyset$ and $|G^{-1}| = |G|$).

aux: Is S bits long.

$f(Q')$: This is a list of Q' elements in $[N]$ and can be encoded using $|Q'| \log N$ bits, but we'll encode it with less if T_g is large.

Summing up we get

$$\begin{aligned} & |\text{Enc}(r, \text{aux}, f)| \\ &= \underbrace{|G| \log(2e/\epsilon\delta) + \log N}_{\text{encoding of } G} + \underbrace{|Q'| \log N}_{f(Q')} + \underbrace{|G| \log(eT_g)}_{(|q'_1|, \dots, |q'_{|G|}|)} + \underbrace{(N - |G| - |Q'|) \log N}_{f([N] - \{G^{-1} \cup Q'\})} + \underbrace{S}_{\text{aux}} \\ &= |G| \log(2e^2 T_g / \epsilon\delta) + (N - |G| - |Q'|) \log N + |Q'| \log N + S + \log N \end{aligned}$$

using the assumption $T_g \leq T \leq \epsilon\delta N/40$ in the statement of the lemma, which in turn implies $\log(2e^2 T_g / \epsilon\delta) \leq \log(N) - 1$, we get

$$\begin{aligned} &\leq |G|(\log N - 1) + (N - |G| - |Q'|) \log N + |Q'| \log N + S + \log N \\ &= (N - |Q'|) \log N + |Q'| \log(N) - |G| + S + \log N \\ &= N \log N - |G| + S + \log N \end{aligned} \tag{10}$$

plugging in the bound from eq.(8) for $|G|$ we get

$$|\text{Enc}(r, \text{aux}, f)| \leq N \log N - \frac{N\delta\epsilon}{2T_g} + S + \log N$$

proving eq.(6) in the Lemma.

5.2 Improved bound if $T_g > 12\delta T$.

We'll now prove a bound on the length of the encoding as stated in eq.(7) in the lemma. Here we assume that T_g is large, i.e., $T_g > 12\delta T$. The bound on the length of the encoding improves the expression we got without this assumption, i.e., eq.(6), by $\delta\epsilon N$ bits. We will achieve this by improving the length of the encoding of $f(Q')$ from the trivial $|Q'| \log(N)$ to $|Q'| \log(N) - \delta\epsilon N$ by exploiting the fact that now a large fraction of the elements in $f(Q')$ falls into the sparse set $[\delta N]$, i.e.,

Claim 5.1 *If $T_g > 12\delta T$, $f(Q')$ can be encoded using $|Q'| \log N - \delta\epsilon N$ bits.*

With this claim we can improve eq.(10) to $(N - |Q'|) \log N + |Q'| \log(N) - |G| + S + \log N$, which now gives eq.(7) from the Lemma.

It remains to prove the claim. By eq.(8) and eq.(9) $|G| \geq \delta\epsilon N/2T_g$, $|Q'| \leq T \cdot |G|$ and $|B| \geq \delta\epsilon N/2$, which implies

$$|B|/|Q'| \geq \delta\epsilon N/2T \cdot |G| \geq T_g/T \geq 12\delta$$

I.e., a 12δ fraction of the queries made during decoding falls into $B \subset [\delta N]$. Using this with Fact 2 we can encode which of the $|B|$ queries from Q' map into B using $|B| \log(e/12\delta)$ bits. For any $x \in B$, we can encode $f(x)$ using $\log(N) - \log(1/\delta)$ bits as $f(x) \in [\delta N]$.

We can now encode $f(Q')$ by first encoding the positions of B in Q' , and then $f(B)$ and $f(Q' \setminus B)$ separately, which requires

$$\begin{aligned}
& (|Q'| - |B|) \log N + |B| (\log(e/10\delta) + \log(N) - \log(1/\delta)) \\
&= |Q'| \log N + |B| \log(e/12) \\
&\leq |Q'| \log N - |B| \cdot 2 \\
&\leq |Q'| \log N - \delta\epsilon N
\end{aligned}$$

bits as claimed.

6 Conclusion and Open Problems

In this work we showed that the known time-space tradeoff $S \cdot T \geq N$ for inverting random functions or permutations $f : [N] \rightarrow [N]$ also holds for truncated outputs almost up to the point where S is big enough to store the entire truncated function table. This shows that the general idea of truncating cryptographic primitives as suggested in [BCKK22] is secure even when preprocessing is considered (as long as the truncated function table is big enough so it can't be stored in practice).

While in this paper we only considered one-wayness of random functions, we believe that our result can be adapted to time-space lower bounds for other primitives, showing the bounds apply also for their truncated analogues. An interesting example considered in [BCKK22] is the discrete logarithm problem, for which a $S \cdot T^2 \geq N$ time-space lower bound is known [CK18].

A likely more challenging but particularly interesting case is the adaption of the “beyond Hellman” proofs of space from [AAC⁺17b] to the truncated setting. The key primitive in [AAC⁺17b] are functions $[N] \rightarrow [N]$ for which a time-space lower bound of $S^k \cdot T \geq N^k$ (for any constant k) can be proven. This improves on the $S \cdot T \geq N$ lower bound for “normal” functions, and the reason this doesn't contradict known upper bounds (by rainbow tables) is the fact that those functions cannot be efficiently evaluated in forward direction (but their entire function table can be computed in time N). A proof that truncation is secure for these functions would allow for better security. Currently those functions are deployed in the proof of space underlying the `chia.net` blockchain, but there's a trade-off that farmers (who are supposed to dedicate space analogous to miners dedicating computation in Bitcoin) can make: by dropping a few bits of every entry, they save space at the cost of having to do some extra computation when computing the proofs². One could use a truncated version of those functions, which would make the initialization of the space for the farmers somewhat more costly, but it would make such “bit dropping” attacks much less attractive.

References

- [AAC⁺17a] Hamza Abusalah, Joël Alwen, Bram Cohen, Danylo Khilko, Krzysztof Pietrzak, and Leonid Reyzin. Beyond hellman's time-memory trade-offs with applications to proofs of space. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part II*, volume 10625 of *LNCS*, pages 357–379. Springer, Heidelberg, December 2017.
- [AAC⁺17b] Hamza Abusalah, Joël Alwen, Bram Cohen, Danylo Khilko, Krzysztof Pietrzak, and Leonid Reyzin. Beyond hellman's time-memory trade-offs with applications to proofs of space. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 357–379. Springer, 2017.

²<https://github.com/Chia-Network/drplotter>

- [BCKK22] Foteini Baldimtsi, Konstantinos Chalkias, Panagiotis Chatzigiannis, and Mahimna Kelkar. Truncator: Time-space tradeoff of cryptographic primitives. Cryptology ePrint Archive, Paper 2022/1581, 2022.
- [CK18] Henry Corrigan-Gibbs and Dmitry Kogan. The discrete-logarithm problem with preprocessing. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 415–447. Springer, 2018.
- [DTT10] Anindya De, Luca Trevisan, and Madhur Tulsiani. Time space tradeoffs for attacks against one-way functions and PRGs. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 649–665. Springer, Heidelberg, August 2010.
- [FN91] Amos Fiat and Moni Naor. Rigorous time/space tradeoffs for inverting functions. In Cris Koutsougeras and Jeffrey Scott Vitter, editors, *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing, May 5-8, 1991, New Orleans, Louisiana, USA*, pages 534–541. ACM, 1991.
- [GT00] Rosario Gennaro and Luca Trevisan. Lower bounds on the efficiency of generic cryptographic constructions. In *41st FOCS*, pages 305–313. IEEE Computer Society Press, November 2000.
- [Hel80] Martin E. Hellman. A cryptanalytic time-memory trade-off. *IEEE Trans. Inf. Theory*, 26(4):401–406, 1980.
- [Oec03] Philippe Oechslin. Making a faster cryptanalytic time-memory trade-off. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 617–630. Springer, 2003.
- [Wee05] Hoeteck Wee. On obfuscating point functions. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 523–532. ACM Press, May 2005.
- [WY21] Shun Watanabe and Kenji Yasunaga. Bit security as computational cost for winning games with high probability. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 161–188. Springer, 2021.
- [Yao90] Andrew Chi-Chih Yao. Coherent functions and program checkers (extended abstract). In *22nd ACM STOC*, pages 84–94. ACM Press, May 1990.